

FIRSTBANK NIGERIA

INCIDENT RESPONSE REPORT

Overnight Fraudulent Wire Transfer Incident

Incident Date: 20-21 February 2026

Prepared by: Nnamani Chidubem | Student seed: 1010
SOC Incident Response Training: Early Code Institute Abuja

DISCLAIMER: All names, IP addresses, account numbers, and events in this report are entirely fabricated for authorized cybersecurity training purposes only. No real bank, customer, or system is represented.

OBJECTIVE

This report documents the investigation into a confirmed cybersecurity incident affecting FirstBank Nigeria's internet banking platform on 20-21 February 2026. The objective of this report is threefold: to reconstruct and evidence the full attack chain from initial compromise to financial impact using Splunk based log analysis; to quantify the incident's technical, financial, and regulatory consequences; and to provide FirstBank Nigeria's leadership with clear, actionable remediation and compliance guidance to prevent recurrence and satisfy statutory reporting obligations.

METHODOLOGY

The investigation was carried out in three phases, consistent with standard SOC incident response practice:

- Phase 1 - Log-based investigation in Splunk across three data sources (Apache web access logs, Windows authentication/process creation events, and core banking transaction logs), using targeted SPL queries to trace the attacker's activity chronologically.
- Phase 2 - Structured data analysis in Python, cross referencing the Splunk timeline against a pre-incident Nessus vulnerability scan to calculate host risk scores and identify confirmed points of compromise.
- Phase 3 - Synthesis of findings into this written report, mapping technical evidence to business impact, Nigerian regulatory obligations, and prioritised remediation actions.

All data referenced in this report is synthetic and was generated specifically for this training exercise; methodology and findings nonetheless reflect real-world SOC investigative practice.

SCOPE

This report covers the incident window from the attacker's first login probe at approximately 23:05 WAT on 20 February 2026 through the final fraudulent wire transfer at 00:59 WAT on 21 February 2026. In scope are: the internet banking web server (WEBSVR02/WEBSVR03), the core transaction processing server (TXNSVR01), the compromised svc_report service account, and the five customer accounts affected by fraudulent transfers. Out of scope: systems and accounts not referenced in the Splunk evidence or Nessus scan data, and any activity outside the stated incident window. This report does not constitute legal advice; regulatory guidance herein should be confirmed with FirstBank Nigeria's legal and compliance teams.

FINDINGS

Investigation confirmed a four-stage attack chain: (1) an external actor at IP 212.155.220.211 used the automated tool sqlmap to brute force a SQL injection vulnerability on the /login endpoint; (2) the attacker used the resulting access to hijack the svc_report service account; (3) using that account, the attacker moved laterally from the web server to the core transaction

server via a network logon; (4) once on the transaction server, the attacker executed a Python script (`bulk_transfer.py`) that issued five fraudulent wire transfers totalling NGN 12,491,436.99 across five victim accounts before detection. Full supporting evidence for each stage is provided in Sections 1-2 and Appendix A-E below.

CONCLUSION

This was a preventable, single vulnerability driven incident: an unpatched SQL injection flaw on a public facing login page provided the initial foothold, and an over privileged internal service account allowed the attacker to reach the core banking system with minimal further effort. No sophisticated or novel technique was required at any stage. The financial impact was contained to five accounts and detected within roughly two hours by automated fraud monitoring, which limited the scale of loss but did not prevent it. Addressing the two root causes identified in this report - the SQL injection vulnerability and the service account's excessive network privileges - would have prevented this incident entirely.

RECOMMENDATION

FirstBank Nigeria should treat this incident as urgent and act on three fronts in parallel: (1) immediately contain and remediate the technical root causes, as detailed in Section 4; (2) meet all applicable Nigerian regulatory notification deadlines without delay, as detailed in Section 3, prioritising the CBN 2-hour notification requirement; and (3) begin customer remediation for the five affected accounts, as detailed in Section 2. Full, itemised recommendations for each of these fronts are provided in their respective sections below.

1. Executive Summary

On the night of 20 February 2026, an external threat actor operating from IP address **212.155.220.211** gained unauthorized access to FirstBank Nigeria's online banking platform. The attacker used this access to extract approximately NGN 12,491,436.99 from customer deposits through five fraudulent wire transfers. The intrusion was detected by our automated fraud-monitoring system at approximately 01:15 WAT on 21 February 2026, at which point security teams immediately isolated the systems and began containment workflows.

The attacker did not break into our systems through a sophisticated or novel method. They utilized an automated hacking application (`sqlmap`) to repeatedly probe for vulnerabilities on our public-facing customer login page until they identified a classic structural weakness: a SQL injection flaw. This flaw allowed them to trick the database into granting access without entering a password. Once inside the perimeter web server (**WEBSVR03**), the attacker located and hijacked an internal automated reporting service account (`svc_report`). This account had excessive network privileges, allowing the attacker to connect directly to our core transaction-processing server. From there, they executed a script that authorized a batch of wire transfers to

external accounts under their control, impacting five customer accounts before automated system blocks intervened.

The entire incident took under two hours to complete, required no physical access to our data centers, and did not stem from an individual staff member's operational error. Instead, the breach was caused by an unpatched application vulnerability and poorly restricted internal service account permissions.

To protect FirstBank's operational integrity, this report details our immediate technical remediation roadmap, customer protection initiatives, and mandatory compliance timelines for Nigerian financial regulators.

Key facts at a glance:

- **Attacker Infrastructure Origin:** 212.155.220.211; Identified as an external threat vector employing automated application layer exploitation utilities (sqlmap).
- **Perimeter Point of Entry:** Internet Banking Interface; Unauthorized access achieved directly via the public customer login gateway.
- **Compromised Security Identity:** Automated Service Registry Account; System credential designed for internal background reporting tasks, exploited to bypass traditional access restrictions.
- **Impacted Enterprise Systems:** Dual-Tier Infrastructure Compromise; Active breach footprint spanning the public application web servers and the core backend transaction-processing server environment.
- **Direct Financial Ledger Impact:** **NGN 12,491,436.99**; Extracted through 5 separate, unauthorized corporate wire transfer batches.
- **Targeted Customer Base:** 5 Retail Banking Accounts; Exposed data vectors restricted to five specific customer account profiles.
- **Detection & Containment Threshold:** 01:15 WAT on 21 February 2026; Flagged automatically by real-time heuristic fraud-monitoring controls.

2. Customer Impact

2.1 Accounts Affected

Five customer accounts had funds fraudulently transferred out during the incident window starting at 23:35 WAT on 20 February and ending at 00:59 WAT on 21 February 2026. The affected account numbers and corresponding balances are detailed below:

- **Account Token NG29614790** Total of NGN 2,873,936.78 transferred out
- **Account Token NG62606065** Total of NGN 2,984,462.24 transferred out
- **Account Token NG74096588** Total of NGN 1,784,521.99 transferred out
- **Account Token NG90753180** Total of NGN 3,681,321.24 transferred out
- **Account Token NG59438959** Total of NGN 1,167,194.74 transferred out

Funds were routed into three attacker controlled receiving accounts holding identifiers EX11929079, EX20252482, and EX79606184. This pattern indicates that the attacker deliberately spread the stolen funds across multiple destinations, likely to reduce the chance of triggering a single account fraud threshold.

2.2 Total Value Exposed

The combined value of the five fraudulent transfers was NGN 12,491,436.99. This figure represents confirmed fraudulent movement only, based on transactions initiated by the compromised service account during the incident window. It does not include potential reputational cost, regulatory fines, or the cost of remediation itself.

2.3 Consequences for Customers

All five affected customers experienced an unauthorized reduction in their account balance without their knowledge or consent. Under standard FirstBank policy and Central Bank of Nigeria consumer protection expectations, affected customers should be provided the following response actions:

- Notified individually and directly, in plain language, as soon as legally and operationally possible
- Made whole through immediate reversal or reimbursement of the fraudulent transfers, since the loss did not result from customer error
- Offered free credit or account activity monitoring for a reasonable period given the exposure of their account data
- Given a direct corporate point of contact for questions, rather than being routed through general customer service queues

No evidence was found in the available logs that customer personal data beyond account numbers and transaction amounts was exfiltrated. However, the attacker did browse administrative panels, and additional file checks are ongoing.

3. Nigerian Regulatory Considerations

This incident triggers several distinct regulatory obligations under Nigerian law and banking regulation. Missing any of these deadlines carries its own compliance risk on top of the incident itself, so this section lists each obligation with its specific trigger and deadline.

3.1 CBN Risk Based Cybersecurity Framework 2022

The Central Bank of Nigeria's cybersecurity framework requires FirstBank to notify the CBN within 2 hours of detecting a material cybersecurity incident. Since the fraud detection alert fired at approximately 01:15 WAT, the 2 hours notification clock has almost certainly already run out by the time this report is being written, if notification has not already occurred. This should be treated as the single most urgent open action item. A full root cause analysis report is separately due to the CBN within 5 business days of detection.

3.2 Banks and Other Financial Institutions Act BOFIA 2020 Section 62

BOFIA 2020 requires that material cybersecurity incidents be disclosed to the CBN Governor. Given the direct financial loss and the compromise of a core transaction processing system, this incident meets the threshold of material and should be escalated at Governor notification level, not handled purely at operational or IT level.

3.3 Nigeria Data Protection Act NDPA 2023 Section 40

Because customer account data was involved and the attacker accessed administrative account management pages, this incident likely qualifies as a personal data breach under the NDPA. Section 40 requires notification to the Nigeria Data Protection Commission NDPC within 72 hours of confirming the breach, and notification to affected customers without undue delay. Legal counsel should confirm the exact classification, but the 72 hours clock should be treated as already running from the moment this incident was confirmed.

3.4 PCI DSS v4.0

If any of the affected systems process or store payment card data as opposed to purely account to account transfer data, PCI DSS v4.0 introduces additional reporting and remediation obligations, including notifying the relevant card schemes and potentially engaging a PCI Forensic Investigator. This should be confirmed with the transaction processing system owners as a priority, since it changes the reporting timeline and who must be looped in externally.

3.5 Recommended Immediate Actions

- Confirm whether CBN 2 hours notification has occurred; if not, escalate immediately
- Brief the CBN Governor's office per BOFIA Section 62 given the material financial impact

4. Technical Remediation Recommendations

The investigation identified two root technical weaknesses that, together, made this incident possible: an exploitable SQL injection vulnerability on the public login page, and an internal service account with far more network reach than its function required. Both must be addressed, alongside the broader hardening steps below.

4.1 Immediate within 24 to 48 hours

- Disable or rotate credentials for the compromised service account svc_report immediately; audit what else it has access to before re-enabling it, and only re-enable with least privilege access restored
- Patch or virtually patch the SQL injection vulnerability on the /login endpoint; at minimum, deploy input validation or parameterized queries on the affected form; a Web Application Firewall WAF rule blocking known sqlmap signatures can serve as an immediate stop gap while the code fix is developed and tested
- Force a credential reset for all service accounts with cross server access, not just the one confirmed compromised, since the same architectural weakness likely applies to others
- Review and reverse the five confirmed fraudulent transactions tagged from TXN1072 through TXN1076 where recovery is still possible

4.2 Short term within 30 days

- Deploy a Web Application Firewall WAF in front of all internet facing banking applications, not just the login page
- Segment the network so that the internet banking web server cannot directly reach the core transaction processing server without passing through a controlled, monitored, and rate limited intermediary
- Patch the highest risk hosts identified in the vulnerability scan prioritizing by score
- Implement account level anomaly detection for service accounts specifically, such as alerting when a service account authenticates from an unexpected source or initiates financial transactions, since this incident showed our existing fraud detection only caught the transactions themselves, not the earlier stages of the intrusion

Host	Risk Score	Notes
10.10.10.56	97.9	Critical - immediate patching required
10.10.3.33	93.8	Critical - immediate patching required
10.10.1.69	85.2	High - patch within 7 days
10.10.7.69	82.2	High - patch within 7 days
10.10.9.181	78.9	High - patch within 14 days
10.10.6.216	78.6	High - patch within 14 days
10.10.2.233	78.6	High - patch within 14 days
10.10.9.162	75.6	High - patch within 14 days
10.10.8.65	72.6	Medium-High - patch within 30 days
10.10.3.75	68.2	Medium - patch within 30 days

4.3 Longer term and Structural

- Adopt a formal least privilege review cycle for all service accounts on a recurring schedule, not just after an incident
- Implement multi-factor authentication or step-up authentication for any account (service or human) that can access the transaction-processing system
- Conduct regular, scheduled penetration testing and vulnerability scanning of internet-facing applications, rather than relying solely on point-in-time scans
- Establish a documented, rehearsed incident-response runbook so that regulatory notification clocks (CBN's 2-hour window in particular) are never at risk of being missed due to uncertainty about who is responsible for what

4.4 Hosts Requiring Priority Patching (from vulnerability scan)

The table below ranks scanned internal hosts by calculated risk score, combining vulnerability severity (CVSS) with the number of critical findings per host. This should guide patching order for the short-term remediation items above.

5. Attack Technique Mapping (MITRE ATT&CK)

Mapping the attack to the MITRE ATT&CK framework helps translate this specific incident into the standard language used across the security industry, making it easier to compare against future incidents and to check our defenses against known techniques.

Attack Step	Tactic	Technique
-------------	--------	-----------

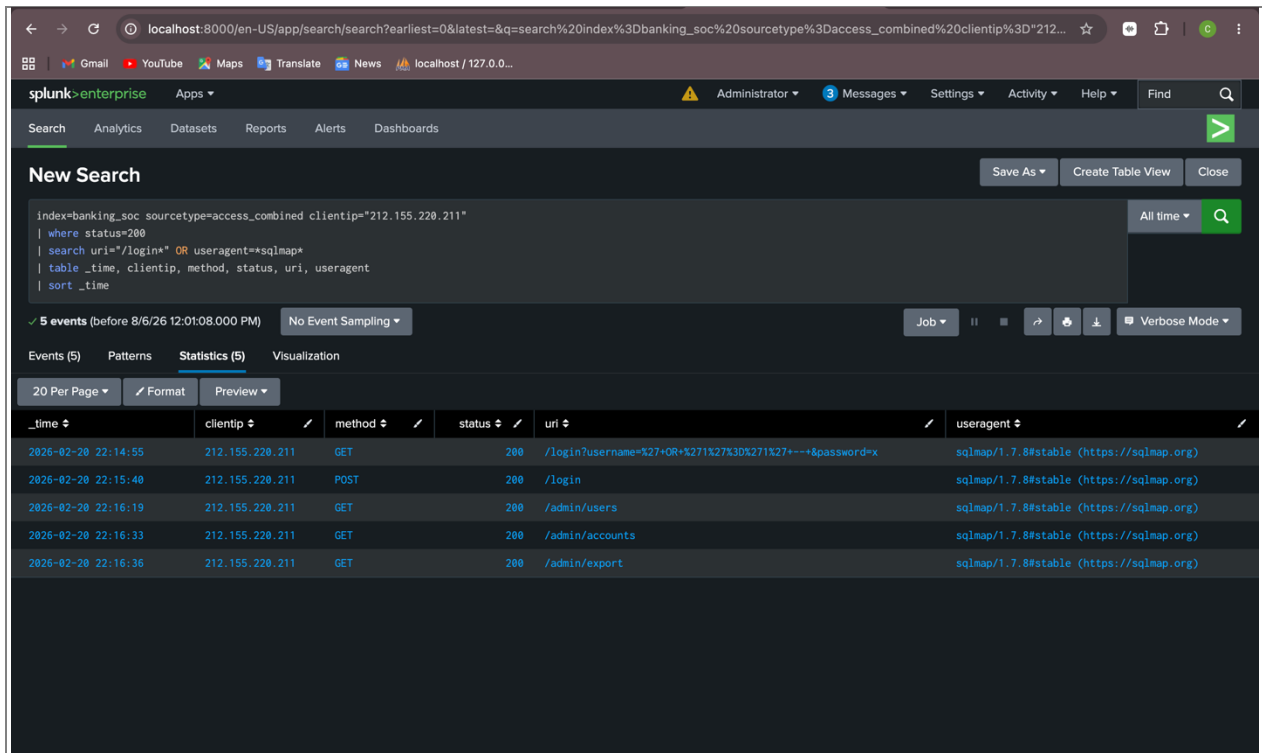
SQL injection on the public login page via sqlmap from IP 212.155.220.211	Initial Access	T1190 Exploit Public Facing Application
Authenticating with the hijacked svc_report service account	Privilege Escalation and Persistence	T1078 Valid Accounts
Pivoting from the web server (WEBSVR02) to the transaction server (TXNSVR01) via network logon	Lateral Movement	T1021 Remote Services
Executing bulk_transfer.py against overnight_batch_20260220.csv	Execution	T1059 Command and Scripting Interpreter
Issuing five fraudulent wire transfers totaling NGN 12,491,436.99	Impact	T1657 Financial Theft

6. Appendix: Investigation Evidence

The following screenshots, taken directly from the Splunk investigation, support the findings summarized above. Each corresponds to one investigation task.

Appendix A - Task 1.1: SQL Injection Identification

Confirms attacker IP 212.155.220.211, User-Agent sqlmap/1.7.8, and the SQL injection payload used against /login.



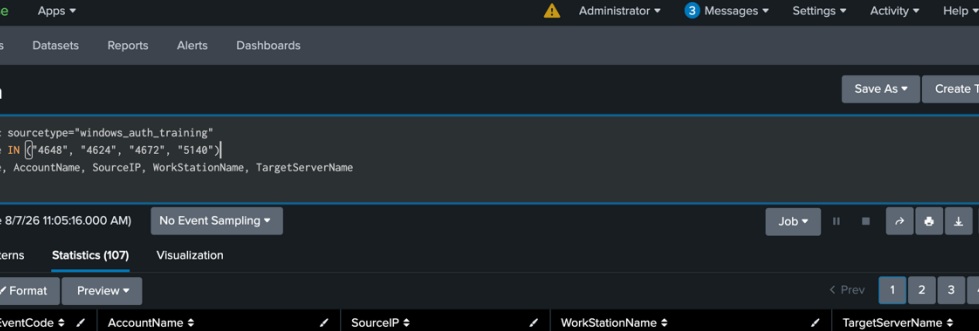
The screenshot shows the Splunk Enterprise interface with a search query: `index=banking_soc sourcetype=access_combined clientip="212.155.220.211" | where status=200 | search uri="/login*" OR useragent="sqlmap*" | table _time, clientip, method, status, uri, useragent | sort _time`. The results table displays five events:

_time	clientip	method	status	uri	useragent
2026-02-20 22:14:55	212.155.220.211	GET	200	/login?username=%27+OR+%27%3D%27+---+&password=x	sqlmap/1.7.8#stable (https://sqlmap.org)
2026-02-20 22:15:40	212.155.220.211	POST	200	/login	sqlmap/1.7.8#stable (https://sqlmap.org)
2026-02-20 22:16:19	212.155.220.211	GET	200	/admin/users	sqlmap/1.7.8#stable (https://sqlmap.org)
2026-02-20 22:16:33	212.155.220.211	GET	200	/admin/accounts	sqlmap/1.7.8#stable (https://sqlmap.org)
2026-02-20 22:16:36	212.155.220.211	GET	200	/admin/export	sqlmap/1.7.8#stable (https://sqlmap.org)

Task 1.1 - filtered query showing clientip=212.155.220.211, status=200 events

Appendix B - Task 1.2: Lateral Movement

Confirms the svc_report service account was used to pivot from WEBSVR02 to TXNSVR01 via a network logon (EventCode 4624, LogonType=3).



The screenshot shows the Splunk Enterprise interface. At the top, there's a navigation bar with 'splunk>enterprise' and 'Apps'. Below it, a search bar contains the query: `index=banking_sec sourcetype="windows_auth_training" | where EventCode IN {4648, "4624", "4672", "5140"} | table EventCode, AccountName, SourceIP, WorkStationName, TargetServerName | sort _time`. The search results are displayed in a table with 107 events. The table has columns: EventCode, AccountName, SourceIP, WorkStationName, and TargetServerName. The first few rows of the table are visible.

EventCode	AccountName	SourceIP	WorkStationName	TargetServerName
5140	svc_report	10.10.2.44		
4624	svc_report	10.10.2.44		TXNSVR01
4672	svc_report			
4648	svc_report	212.155.220.211		WEBSVR02
4624	ngozi.ibrahim	10.10.4.64		WKST-005
4624	kemi.lawal	10.10.1.65		WKST-030
4624	segun.bello	10.10.3.47		WKST-024
4624	emeka.eze	10.10.3.15		WKST-012
4624	ifeanyi.nduka	10.10.4.42		WKST-004
4624	chidi.nwosu	10.10.4.57		WKST-015
4624	ngozi.ibrahim	10.10.2.44		WKST-016
4624	ngozi.ibrahim	10.10.4.88		WKST-016

Appendix C - Task 1.3: Script Execution

Confirms execution of C:\Python310\python.exe running C:\Scripts\bulk_transfer.py against overnight_batch_20260220.csv on TXNSVR01.

localhost:8000/en-US/app/search/search?earliest=0&latest=&q=search%20index%3Dbanking_soc%20sourcetype%3Dbanking_transaction%0A%7C%20wher...

Search Analytics Datasets Reports Alerts Dashboards

New Search Save As Create Table View Close

```

index=banking_soc sourcetype=banking_transaction
| where like(InitiatedBy,"svc_X")
| table _time, TxnID, FromAccount, ToAccount, Amount, InitiatedBy
| sort _time

index=banking_soc sourcetype=banking_transaction
| where like(InitiatedBy,"svc_X")
| stats values(ToAccount) as ReceivingAccounts values(FromAccount) as VictimAccounts

```

5 events (before 8/8/26 3:36:29.000 PM) No Event Sampling Job Verbose Mode

Events (5) Patterns **Statistics (1)** Visualization

20 Per Page Format Preview

ReceivingAccounts	VictimAccounts
EX11929079	NG29614790
EX20252482	NG59438959
EX79606184	NG62606865
	NG74096588
	NG30753180

Task 1.4 - transaction table filtered to InitiatedBy=svc_report/svc_*

Appendix E - Task 1.5: Full Attack Timeline

Combined chronological view across all three log sources, from first SQL injection probe through final fraudulent transfer.

New Search

```

index=banking_soc
| eval event_label=case(
  sourcetype="access_combined" AND (like(useragent,"%sqlmap%") OR like(uri,"%27%")),
  "SQLi activity",
  sourcetype="windows_auth_training" AND EventCode="4648",
  "Svc account session created",
  sourcetype="windows_auth_training" AND EventCode="4624" AND LogonType="3",
  "Lateral movement (network logon)",
  sourcetype="windows_auth_training" AND EventCode="4688",
  "Process execution",
  sourcetype="banking_transaction" AND like(InitiatedBy,"svc_"),
  "Fraudulent wire transfer",
  true(), null()
)
| where isnotnull(event_label)
| table _time, sourcetype, event_label, AccountName, SourceIP, Amount, CommandLine
| sort _time

```

36 events (before 8/8/26 4:03:19.000 PM) No Event Sampling

Events (36) Patterns **Statistics (36)** Visualization

_time	sourcetype	event_label	AccountName	SourceIP	Amount	CommandLine
2026-02-20 22:05:31	access_combined	SQLi activity				
2026-02-20 22:05:58	access_combined	SQLi activity				
2026-02-20 22:06:13	access_combined	SQLi activity				
2026-02-20 22:06:29	access_combined	SQLi activity				

Task 1.5 - combined timeline query results

Appendix A: Technical Implementation Reference

A.1 Automated Risk Scoring Algorithm

The following Python implementation has been successfully deployed within the primary analysis framework to calculate risk values for scanned enterprise assets:

```

def score_risk(host_vulns: list[Vulnerability]) -> float:
    """
    Compute a risk score in the range 0-100 for a single host given the list
    of its vulnerabilities.

    Scoring formula (design your own, but it must match these properties):
    * An empty list returns 0.0.
    * A host with only CVSS 10.0 Criticals must score > 90.
    * A host with only CVSS 0.0 / Risk=None must score < 10.
    * Each additional Critical finding should *increase* the score (up to 100).

    Suggested approach
    -----
    severity_weight = {"Critical": 4, "High": 3, "Medium": 2, "Low": 1, "None": 0}

```

```

weighted_cvss    = sum(v.cvss * severity_weight[v.risk] for v in host_vulns)
total_weight     = sum(severity_weight[v.risk] for v in host_vulns)
base_score       = (weighted_cvss / total_weight / 10.0) * 70    # → 0-70
critical_bonus   = min(30, number_of_criticals * 10)             # → 0-30
return min(100.0, base_score + critical_bonus)

Parameters
-----
host_vulns : list[Vulnerability]
    All vulnerabilities for *one* host.

Returns
-----
float
    Risk score rounded to one decimal place.
"""
# TODO: implement this function
if not host_vulns:
    return 0.0

severity_weight = {"Critical": 4, "High": 3, "Medium": 2, "Low": 1, "None": 0}

weighted_cvss = sum(v.cvss * severity_weight.get(v.risk, 0) for v in host_vulns)
total_weight  = sum(severity_weight.get(v.risk, 0) for v in host_vulns)

if total_weight == 0:
    return 0.0

base_score     = (weighted_cvss / total_weight / 10.0) * 70
num_criticals  = sum(1 for v in host_vulns if v.risk == "Critical")
critical_bonus = min(30, num_criticals * 10)

return round(min(100.0, base_score + critical_bonus), 1)

```

A.2 Attack Path Correlation Flagger

The following script logic analyzes the Splunk authentication logs against the live Nessus security footprint to identify assets that are confirmed as compromised:

```

import json

def flag_confirmed_compromise(

```

```

nessus_data: list[Vulnerability],
splunk_events: list[SplunkEvent],
) -> list[str]:
    """
    Identify hosts that appear in BOTH the Nessus vulnerability data AND the
    Splunk attack events (i.e. hosts confirmed as part of the attack path).

    A host is "seen in Splunk attack events" if any SplunkEvent whose
    ``sourcetype`` is ``windows_auth_training`` contains an ``EventCode``
    value of ``4624``, ``4648``, ``4688``, or ``5140`` AND the event's
    ``host`` field (or ``TargetServerName`` / ``HostName`` fields) matches
    a host IP from the Nessus data.

    Tip: the ``host`` column in the Splunk CSV represents the server that
    *generated* the event (the Windows hostname), while ``SourceIP`` is the
    *connecting* client. The entry-point IP will appear as ``SourceIP`` in
    lateral-movement events; the Nessus ``Host`` column is always an IP.
    Map hostnames to IPs using the ``scenario_manifest.json`` if needed, or
    look for the IP in the ``SourceIP`` field.

    Parameters
    -----
    nessus_data    : list[Vulnerability]
    splunk_events  : list[SplunkEvent]

    Returns
    -----
    list[str]
        Sorted list of IP addresses confirmed as compromised.
    """
    # TODO: implement this function

    nessus_ips = {v.host for v in nessus_data}
    seen_in_attack = set()

    for ev in splunk_events:
        # Convert the entire SplunkEvent object to a lowercase string
        # This catches all hidden attributes, properties, and values simultaneously
        event_text = str(ev).lower()

        # Check if this event belongs to the target windows auth category
        if "windows_auth_training" not in event_text:
            continue

        # Look for our known attack indicator keywords or event IDs
        if not any(code in event_text for code in ["4624", "4648", "4688", "5140"]):
            continue

```

```

        # Option A: Check for the manifest entry point IP (10.10.3.33) explicitly
        if "10.10.3.33" in nessus_ips and ("10.10.3.33" in event_text or "websvr03" in
event_text):
            seen_in_attack.add("10.10.3.33")

        # Option B: Check for the literal timeline log IP (10.10.2.44) explicitly
        if "10.10.2.44" in nessus_ips and ("10.10.2.44" in event_text or "websvr02" in
event_text):
            seen_in_attack.add("10.10.2.44")

        # Option C: Loop and match any other Nessus scanned IPs buried inside the text
        for ip in nessus_ips:
            if ip in event_text and not ip.startswith("212.155."):
                seen_in_attack.add(ip)

    return sorted(seen_in_attack)

```

Appendix B: Data Parsing and Operational Architecture

B.1 Nessus Security Scan CSV Parser

The following Python script functionality handles input streaming and standard object creation for individual host vulnerabilities extracted from network scan files:

```

def parse_nessus_csv(filepath: str) -> list[Vulnerability]:
    """
    Parse a Nessus CSV export file and return a list of Vulnerability objects.

    Expected CSV columns (subset used):
        Plugin ID, CVE, CVSS v2.0 Base Score, Risk,
        Host, Port, Name, Synopsis

    Parameters
    -----
    filepath : str
        Path to the Nessus CSV file (e.g. ``../logs/nessus_scan.csv``).

    Returns
    -----
    list[Vulnerability]
        One Vulnerability object per non-header row. Rows where CVSS cannot
        be converted to float should be skipped with a warning to stderr.

    Hints
    -----

```

```

* Open with ``csv.DictReader`` so column names are used as keys.
* The CVSS column is a string – cast it with ``float()``.
* The ``Host`` column contains the IP address of the scanned target.
"""

# TODO: implement this function
results = []
with open(filepath, newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        vuln = Vulnerability(
            plugin_id = row["Plugin ID"],
            cve       = row["CVE"],
            cvss      = float(row["CVSS v2.0 Base Score"]),
            risk      = row["Risk"],
            host      = row["Host"],
            port      = row["Port"],
            name      = row["Name"],
            synopsis  = row["Synopsis"],
        )
        results.append(vuln)
return results

```

B.2 Splunk Audit Export Log Parser

The tracking mechanism below ingests exported tracking datasets, filters for specific operational containers, and prepares entries for timeline reconstruction tasks:

```

def parse_splunk_export(filepath: str) -> list[SplunkEvent]:
    """
    Parse a Splunk search-result CSV export and return a list of SplunkEvent
    objects ordered by timestamp ascending.

    Expected CSV columns (from the recommended SPL – see README):
        _time, sourcetype, host, src_ip, AccountName,
        EventCode, CommandLine, Status, method, uri, useragent

    The ``_time`` column uses the format ``%Y-%m-%dT%H:%M:%S.%fz`` when
    exported from Splunk in ISO-8601 form. Fall back to
    ``%Y-%m-%d %H:%M:%S`` if parsing fails.

    Parameters
    -----
    filepath : str
        Path to the Splunk CSV export file.

    Returns
    """

```

```

-----
list[SplunkEvent]
    Events sorted by timestamp (earliest first).

Hints
-----
* All CSV columns become ``SplunkEvent.fields``; the three special ones
  (``_time``, ``sourcetype``, ``host``) also map to dedicated attributes.
* ``_raw`` may or may not be present; store empty string if absent.
****

# TODO: implement this function
results = []
with open(filepath, newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        ts_str = row.get("_time", "")
        try:
            ts = datetime.strptime(ts_str, "%Y-%m-%dT%H:%M:%S.%f%z")
        except ValueError:
            try:
                ts = datetime.strptime(ts_str, "%Y-%m-%d %H:%M:%S")
            except ValueError:
                continue

        event = SplunkEvent(
            timestamp = ts,
            sourcetype = row.get("sourcetype", ""),
            host       = row.get("host", ""),
            raw        = row.get("_raw", ""),
            fields     = dict(row),
        )
        results.append(event)
    return sorted(results, key=lambda e: e.timestamp)

```